

Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: IMPLEMENTACIÓN DE ANALIZADORES LÉXICOS EN LEX.

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Resumen

El presente informe muestra ejercicios Lex desarrollados para la solución de distintos problemas comunes en el área de backend y teoría de autómatas. El análisis léxico constituye la primera fase de un sistema traductor, el cual transforma un conjunto de flujo continuo de caracteres provenientes de un código fuente en secuencias organizadas con significado semántico llamados tokens.

2. Estructura de un Archivo LEX

Un archivo fuente lex se encuentra estructurado por los siguientes bloques funcionales separados por (%%):

- **Área de declaración de variables:** Se declaran las librerías y variables que se van a utilizar.
- **Área de expresiones regulares:** En esta se adjuntan todas las expresiones regulares necesarias para cada programa y su respectiva impresión en consola si se llega a encontrar algo relacionado con la misma.
- **Área de código:** En esta se empieza a codificar en el lenguaje previamente seleccionado.

3. Implementación y Código Fuente

Desarrollar un programa Lex capaz de identificar secuencias binarias que cumplan estrictamente con la expresión regular $0(0|1)0^*$, determinando su validez e imprimiendo el resultado.

Pruebas y Ejecución:

```
iccd332@DESKTOP-52QQFVV x + v
(base) iccd332@DESKTOP-52QQFVV:~$ mkdir cinco
(base) iccd332@DESKTOP-52QQFVV:~$ cd cinco
(base) iccd332@DESKTOP-52QQFVV:~/cinco$ nano cinco.l
(base) iccd332@DESKTOP-52QQFVV:~/cinco$ nano entrada.txt
```

```
iccd332@DESKTOP-52QQFVV x + v
GNU nano 7.2 cinco.l
%{
    #include <stdio.h>
    #include <stdlib.h>
}%
%%
0(0|1)0* printf("SECUENCIA VALIDA: %s\n", yytext);
\n
    printf("SECUENCIA NO VALIDA: %s\n", yytext);
.
%%
```

```
iccd332@DESKTOP-52QQFVV x + v
GNU nano 7.2 entrada.txt
00
0100
111
10010
11
10010110
1011011
```

```
iccd332@DESKTOP-52QQFVV x + v
(base) iccd332@DESKTOP-52QQFVV:~/cinco$ lex cinco.l
(base) iccd332@DESKTOP-52QQFVV:~/cinco$ ./a.out < entrada.txt > salida.txt
(base) iccd332@DESKTOP-52QQFVV:~/cinco$ cat salida.txt
SECUENCIA VALIDA: 00
SECUENCIA VALIDA: 0100
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 00
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 0
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 00
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 01
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 0
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 01
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 01
SECUENCIA NO VALIDA: 1
```

```
salida.txt
Archivo Editar Ver H1 v B I ...
SECUENCIA VALIDA: 00
SECUENCIA VALIDA: 0100
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 00
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 0
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 00
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 01
SECUENCIA NO VALIDA: 1
SECUENCIA NO VALIDA: 0
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 01
SECUENCIA NO VALIDA: 1
SECUENCIA VALIDA: 01
SECUENCIA NO VALIDA: 1
```

4. Conclusión

La implementación de este código nos permitió observar que la ejecución es altamente eficiente al reconocer patrones estrictos, como la expresión regular binaria $0(0|1)0^*$.